Extração de Regras de Associação

1

Algoritmos de Mineração de Regras de Associação

- Apriori
- Partition

Um algoritmo é dito escalável se, dada uma quantidade fixa de memória principal, seu tempo de execução cresce linearmente com o número de registros da base de dados.

2

Mineração de Regras de Associação

→ Entrada:

- Base de dados de transações;
- Suporte mínimo (*supmin*);
- Confiança mínima (*confmin*).

→ Saída:

- Todas as regras de associação que possuem suporte e confiança maiores ou iguais ao suporte e à confiança mínimos.

3

Mineração de Regras de Associação

Identificar as regras de associação com SupMin=0.3 e ConfMin=0.8, considerando-se a seguinte base de dados.

ID	Itens
1	{Café, Pão, Manteiga}
2	{Leite, Cerveja, Pão, Manteiga}
3	{Café, Pão, Manteiga}
4	{Leite, Café, Pão, Manteiga}
5	{Cerveja}
6	{Manteiga}
7	{Pão}
8	{Feijão}
9	{Arroz, Feijão}
10	{Arroz}

4

Regras obtidas:

{Café} => {Pão}	Sup=0.3 e Conf=1.0
{Café} => {Manteiga}	Sup=0.3 e Conf=1.0
{Pão} => {Manteiga}	Sup=0.4 e Conf=0.8
{Manteiga} => {Pão}	Sup=0.4 e Conf=0.8
{Café,Pão} => {Manteiga}	Sup=0.3 e Conf=1.0
{Café,Manteiga} => {Pão}	Sup=0.3 e Conf=1.0
{Café} => {Pão,Manteiga}	Sup=0.3 e Conf=1.0

5

Mineração de Regras de Associação

→ Definição:

Um conjunto de itens Z é dito freqüente quando o seu suporte é maior ou igual ao suporte mínimo.

→ Desta forma, podemos dividir o problema de minerar regras de associação em duas fases.

Fase 1: Encontrar cada conjunto freqüente Z (Sup(Z) ≥ SupMin);

Fase 2: Para cada conjunto freqüente Z, identificar seus possíveis subconjuntos X e Y, de tal forma que:

$Z = X \cup Y$ e $Conf(X \Rightarrow Y) \geq ConfMin$.

6

Mineração de Regras de Associação

Fase 1: Identificação dos conjuntos freqüentes.
→ É a fase computacionalmente cara.
→ Para um conjunto de itens de tamanho n, existem 2^n possíveis subconjuntos freqüentes.
→ Dois algoritmos básicos propostos para esta fase:
- *Apriori*
- *Partition*

Fase 2: Identificação das regras a partir dos conjuntos freqüentes.

7

Estratégia *Apriori*

O algoritmo *Apriori* considera as seguintes propriedades com o objetivo de diminuir o espaço de busca, ou seja, evitar que todos os 2^n subconjuntos sejam avaliados.

→ Todo subconjunto de um conjunto freqüente é freqüente.
(Se {A,B,C} é freqüente, então {A,B} é freqüente.)

→ Pela contra-positiva: Todo conjunto que contém um subconjunto não freqüente também não é freqüente.
(Se {A,B} não é freqüente, então {A,B,C} não é freqüente.)

8

Estratégia *Apriori*

• Calcular o suporte de todos os conjuntos de tamanho 1 e, em seguida, eliminar aqueles que não possuem o suporte mínimo.

• Formar todos os possíveis conjuntos de tamanho 2 a partir daqueles de tamanho 1 resultantes do passo anterior. Em seguida, eliminar os novos conjuntos que não possuem o suporte mínimo.

• Repetir o procedimento anterior até que, no k-ésimo passo, nenhum novo conjunto de tamanho k, obtido a partir dos conjuntos de tamanho k-1, tenha suporte mínimo.

9

Estratégia *Apriori*

TID	Itens Comprados
101	leite, pão, suco
792	leite, suco
1130	leite, pão, ovos
1735	pão, biscoito, café

Considerar SupMin = 50% (2 tuplas)

Sup({leite}) = 3 Sup({pão}) = 3 Sup({suco}) = 2 Sup({ovos}) = 1 Sup({biscoito}) = 1 Sup({café}) = 1	Sup({leite, pão}) = 2 Sup({leite, suco}) = 2 Sup({pão, suco}) = 1	Sup({leite, pão, suco}) = 1
Freqüentes: {leite} {pão} {suco}	Freqüentes: {leite, pão} {leite, suco}	Freqüentes: -----

10

Estratégia *Apriori*

1. F_1 = conjuntos freqüentes de tamanho 1;
2. k = 1;
3. **enquanto** ($F_k \neq \emptyset$) **faça**
4. k = k + 1;
5. Gerar C_k (todos os candidatos de tamanho k) a partir de F_{k-1};
6. **para cada** transação t pertencente a base de dados **faça**
7. Incrementar o contador associado a todo candidato C_k cujos itens pertençam a t;
8. F_k = todos os candidatos pertencentes a C_k com suporte mínimo maior ou igual a SupMin;
9. **fim-enquanto**;
10. Resposta = união de todos os conjuntos F_k;

Observe que, em cada uma das k iterações, o algoritmo *Apriori* percorre toda a base de dados.

11

Gerar C_k (todos os candidatos de tamanho k) a partir de F_{k-1};

A estratégia de geração de C_k a partir de F_{k-1} também considera a propriedade de que todo subconjunto de um conjunto freqüente é freqüente. Desta forma, diminui a quantidade de candidatos gerados, eliminando alguns que são garantidamente não freqüentes.

→ Considere que, dentro de cada conjunto, os itens estejam ordenados. Então, o conjunto $\{a_1, a_2, ..., a_{k-2}, a_{k-1}, a_k\}$ só será gerado em C_k se os subconjuntos $\{a_1, a_2, ..., a_{k-2}, a_{k-1}\}$ e $\{a_1, a_2, ..., a_{k-2}, a_k\}$ pertencerem a F_{k-1}.

(O conjunto {1,2,3,4} só poderá ser gerado se {1,2,3} e {1,2,4} forem freqüentes.)

→ Se $\{a_1, a_2, ..., a_{k-2}, a_{k-1}, a_k\}$ for gerado em C_k, será podado se possuir algum subconjunto que não seja freqüente.

(O conjunto candidato {1,2,3,4} será eliminado de C_4 se, por exemplo, {2,3,4} não for um conjunto freqüente.)

12

Gerar C_k (todos os candidatos de tamanho k) a partir de F_{k-1}:

A estratégia de geração de C_k a partir de F_{k-1} divide-se então em duas fases: junção e poda.

Junção: Para cada dois conjuntos $\{a_1, a_2, ..., a_{k-1}\}$ e $\{b_1, b_2, ..., b_{k-1}\}$ de F_{k-1}:
Se $(a_1 = b_1) \wedge (a_2 = b_2) \wedge ... \wedge (a_{k-2} = b_{k-2}) \wedge (a_{k-1} < b_{k-1})$ então
gere o candidato $\{a_1, a_2, ..., a_{k-1}, b_{k-1}\}$ em C_k.

(Se {1,2,3} e {1,2,4} são conjuntos de F_3, então gerar {1,2,3,4} em C_4.)

Poda: Para cada conjunto de C_k, eliminar aqueles que possuem um subconjunto não freqüente.

(O conjunto candidato {1,2,3,4} será eliminado de C_4 se, por exemplo, {2,3,4} não for um conjunto freqüente.)

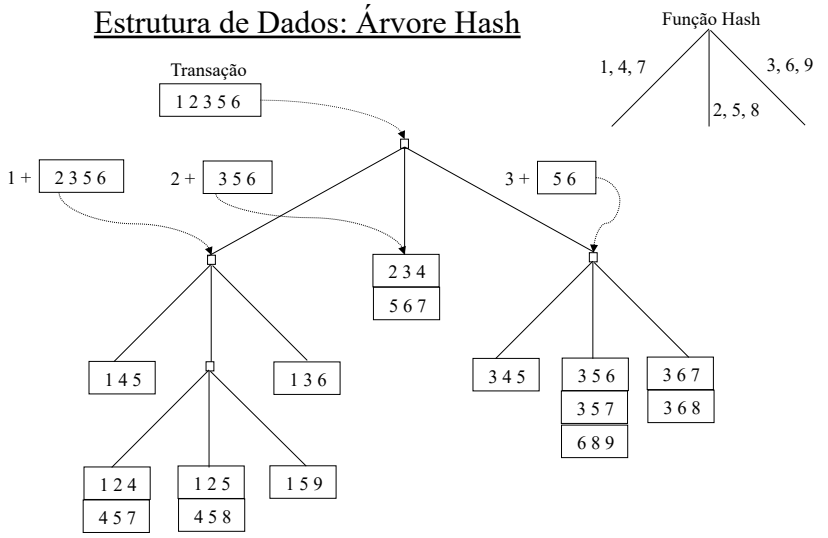
13

Gerar C_k (todos os candidatos de tamanho k) a partir de F_{k-1}:

F_3	Junção	C_4	Poda	C_4 (podado)
{A,B,C}	⇒	{A,B,C,D}	⇒	{A,B,C,D}
{A,B,D}		{B,C,D,E}		
{A,C,D}				
{B,C,D}				
{B,C,E}				

14

Estrutura de Dados: Árvore Hash

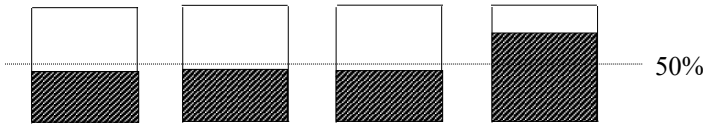


15

Estratégia *Partition*

O algoritmo *Partition* considera a seguinte propriedade com o objetivo de diminuir o número de leituras a toda a base de dados.

→ Considere a base de dados de transações dividida em n partições. Se um conjunto F é freqüente em relação a toda a base de dados (freqüência global), então F é freqüente em relação a pelo menos uma partição (freqüência local), ou seja, possui suporte maior ou igual ao mínimo dentro desta partição.



16

Estratégia *Partition*

A estratégia *Partition* é dividida em duas fases: na primeira, são gerados os conjuntos candidatos e, na segunda, dentre estes são identificados os freqüentes. Em cada fase é realizada (apenas) uma leitura em toda a base de dados.

Fase I:

→ A base de dados é dividida em partições que caibam na memória principal. Para cada partição, são gerados os conjuntos freqüentes locais, utilizando-se as idéias da estratégia *Apriori*.

→ Desta forma, em um único acesso a toda a base de dados, os conjuntos freqüentes locais de cada partição são gerados. Estes conjuntos são os candidatos a freqüentes globais.

Fase II:

→ Todas as transações da base de dados são percorridas para verificar quais freqüentes locais (candidatos globais) são freqüentes globais.

17

Estratégias *Apriori* e *Partition*

→ Na estratégia *Partition*, a base de dados é lida apenas duas vezes.
Na estratégia *Apriori*, a base de dados é lida em cada uma das k iterações.

→ Se, por um lado, a estratégia *Apriori* realiza um número maior de leituras à base de dados, estas várias leituras permitem que, dentre os conjuntos candidatos, apenas os freqüentes passem à iteração seguinte.

→ Na estratégia *Partition*, passam para a última fase e devem ser processados todos os freqüentes locais (candidatos globais), identificados em cada partição. Este fato, dependendo do número de candidatos gerados que não são de fato freqüentes, pode comprometer o desempenho deste algoritmo.

18

Referências (sobre regras de associação):

- Relatório Técnico: “Regras de Associação:
suas Diferentes Formas e Algoritmos de Mineração”,
I.M.B.Mendes, A.Plastino e L.S.Ochi,
Instituto de Computação - UFF - RT-09/02, 2002.
(disponível em http://www.dcc.ic.uff.br/~plastino/PA.HTM)

- Capítulo 5 “Mining Association Rules in Large Databases”
Data Mining: Concepts and Techniques
J. Han & M. Kamber – Second Edition.

Outra referência:

- Relatório Técnico: “Improving Direct Counting for Frequent Set Mining”,
C.Targa, A.Prado e A.Plastino,
Instituto de Computação - UFF - RT-02/03, 2003.
(disponível em http://www.dcc.ic.uff.br/~plastino/PA.HTM)

- Artigo: ”Improving Direct Counting for Frequent Itemset Mining"
A.B.Prado, C.N.Targa e A.Plastino
6th International Conference on Data Warehousing and Knowledge Discovery (DaWaK),
LNCS 3181, Zaragoza, 2004.

Artigos com outros algoritmos/ideias:

“A New SQL-like Operator for Mining Association Rules”

“Mining Association Rules with Item Constraint”

“Mining Generalized Association Rules”

“Mining Frequent Patterns without Candidate Generation”

“Integrating Classification and Association Rule Mining”

“Enhancing the Apriori Algorithm for Frequent Set Counting”

“An Effective Hash-Based Algorithm for Mining Association Rules”